

Ressource R2.02 : Développement d'application avec IHM

JavaFX - Séance TD 3 : initiation à Scene Builder

Lors des séances précédentes, l'IHM était réalisée en déclarant les composants en tant que variables et en programmant la construction de *Scene graph* (cf. méthode `creerScene()`).

Il existe une autre possibilité pour réaliser une IHM : produire un fichier FXML (extension *.fxml*) contenant la structure du *Scene graph* et la description précise de chaque composant. Dans cette séance nous allons utiliser un outil, **JavaFX Scene Builder**¹, qui permet de créer rapidement une IHM et de produire le fichier *fxml* correspondant.

1. Présentation générale

Scene Builder est un outil de construction graphique d'IHM. Il permet la création de *Scene graph* par des glisser/déposer de composants depuis une bibliothèque.

L'écran principal de Scene Builder est composé de plusieurs zones (voir *figure 1*) .

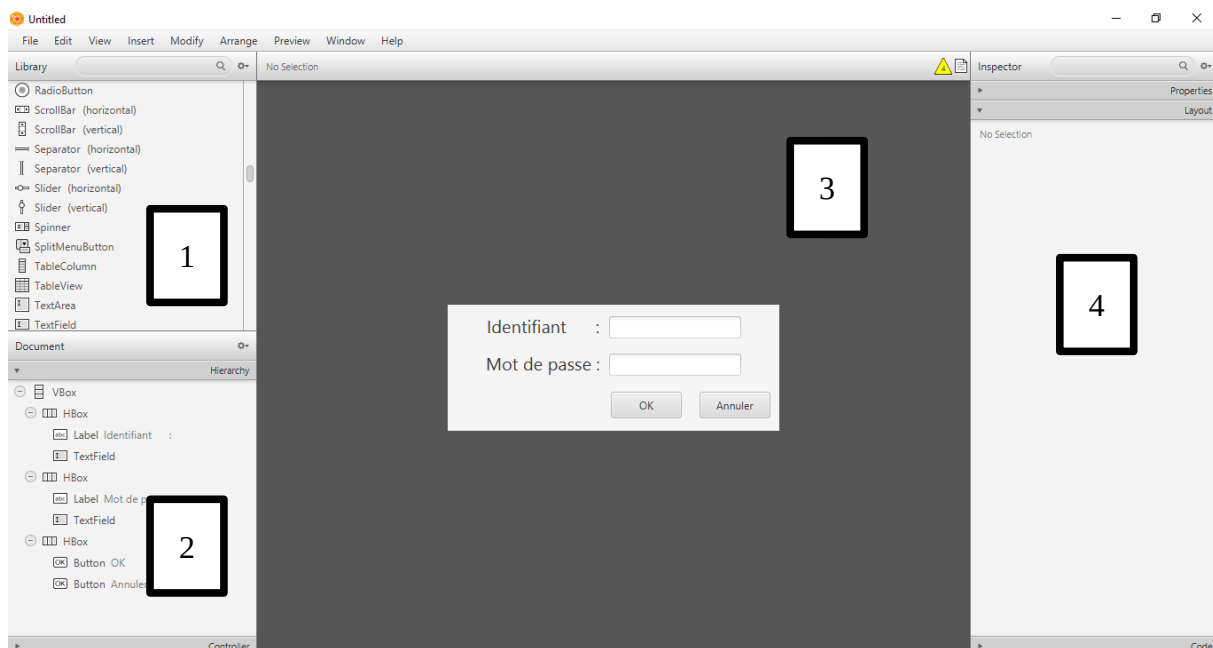


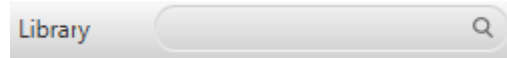
figure 1 : écran principal de Scene Builder

Note : avec Scene Builder, on ne construit pas des fenêtres (*Stage*) mais des *Scene graph*.

¹ que nous appellerons simplement Scene Builder dans la suite du document.

1

En haut à gauche de l'écran se trouve la bibliothèque de composants (*Library*), organisée en plusieurs catégories : *Customs*, *Containers*, *Controls*, etc. Cette zone peut être affichée en mode *Sections* ou en mode *List* grâce à l'icone  .
À noter enfin qu'une barre de recherche facilite la sélection d'un composant :



2

La zone *Document* à gauche en bas de l'écran comporte un volet *Hierarchy*, qui est la représentation du *Scene graph* en cours d'élaboration. Zone très pratique, on peut y sélectionner un composant, en ajouter ou en supprimer (la zone comporte aussi un volet *Controller* sur lequel nous reviendrons plus tard).

3

Cette zone centrale est la zone d'élaboration de l'IHM ; elle contient la représentation graphique du *Scene graph* en cours d'élaboration sur lequel on fait glisser les composants choisis dans la bibliothèque.

4

Enfin, la zone de droite contient tous les paramètres du composant préalablement sélectionné. Cette zone contient trois volets :

- un volet pour les propriétés générales du composant (*Properties*),
- un volet pour les paramètres de taille et de disposition (*Layout*),
- un volet pour le code (*Code*) sur lequel nous reviendrons ultérieurement.

Note : chacune de ces zones peut être affichée ou cachée grâce au menu déroulant *View*.

2. Premier exemple : la fenêtre de choix d'une couleur

Pour ce premier exemple, voici l'IHM à obtenir :



figure 2 : IHM à construire

À FAIRE

Commencez par créer un nouveau fichier et sauvegardez-le (votre fichier prend l'extension fxml).

Avec Scene Builder, on commence toujours la création d'un *Scene graph* par sa racine.

Note : il faut bien réfléchir à cette racine car une fois choisie il est compliqué de la changer.

À FAIRE

Ici la racine est un `BorderPane`. Sélectionnez le `BorderPane` dans la bibliothèque et faites-le glisser sur votre IHM (ou sur votre "hierarchy").

Dans le volet `Layout` de ce composant, fixer sa taille à 350 de large (`Pref Width`) et à 300 de haut (`Pref Height`) Puis ajoutez un `padding` de 10 tout autour.

Ajoutez ensuite les autres composants en vous basant sur le `Scene graph` et les consignes de la page suivante.

Pour les quatre boutons radio, pensez à :

- les inclure dans un même `ToggleGroup`,
- faire en sorte que le bouton `Bleu` soit sélectionné par défaut.

Scene graph :

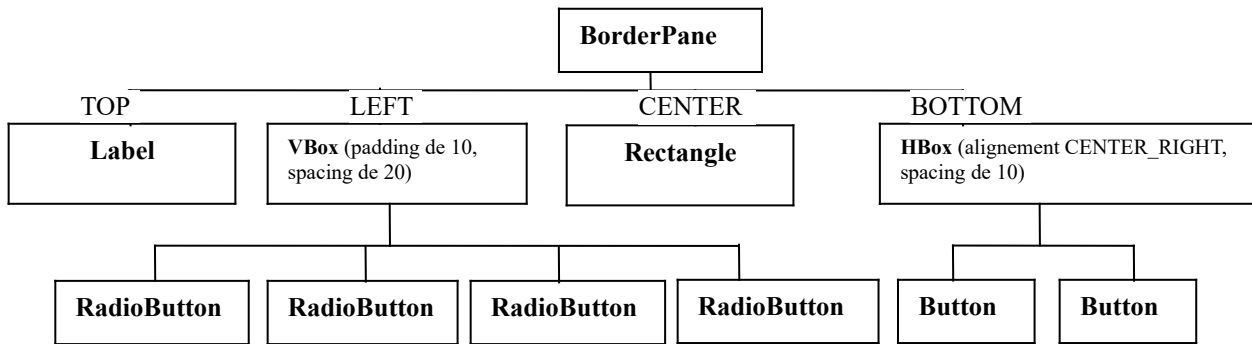


figure 3 : Scene graph de la fenêtre à construire

Consignes :

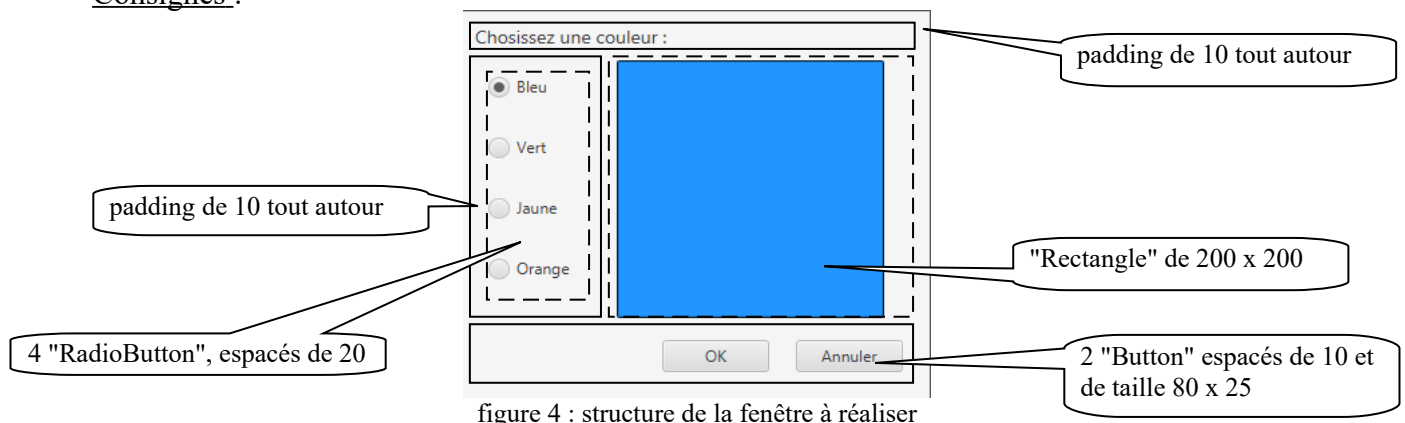


figure 4 : structure de la fenêtre à réaliser

Rappel sur les dimensions d'un composant

Un composant possède une taille par défaut (largeur et hauteur), mais d'autres propriétés peuvent être définies :

- *minWidth* : largeur minimale pour le composant,
- *minHeight* : hauteur minimale pour le composant,
- *prefWidth* : largeur préférée (idéale) du composant,
- *prefHeight* : hauteur préférée (idéale) du composant,
- *maxWidth* : largeur maximale pour le composant,
- *maxHeight* : hauteur maximale pour le composant.

Ici, la fenêtre n'étant pas redimensionnable on se contentera de fixer les *prefWidth* et *prefHeight*. Pour les autres on laissera "USE_COMPUTED_SIZE".

3. Voir le résultat

3.1. Visualisation graphique du résultat

La zone centrale donne un aperçu assez fidèle de l'IHM en cours de construction. Mais pour avoir un aperçu plus réaliste, on utilise le mode Preview (menu **Preview** → **Show preview in Window**). Le *Scene graph* apparaît alors dans une "fenêtre" qui peut être manipulée de manière assez réaliste.

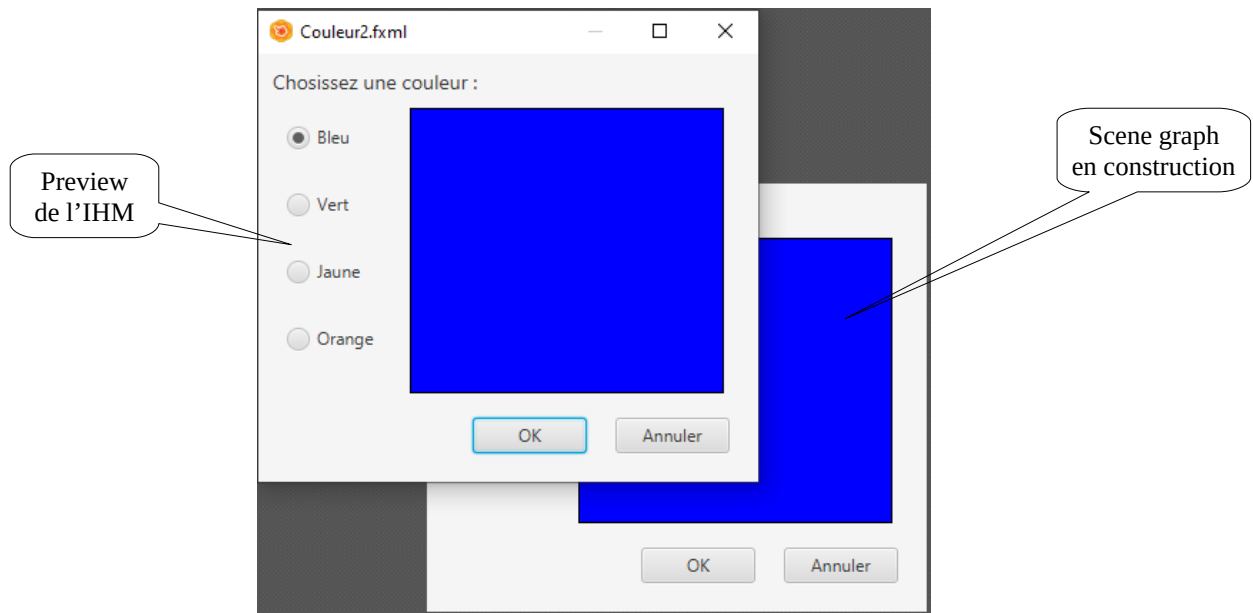


figure 5 : prévisualisation de la scène (mode Preview)

Note : lors du Preview le *Scene graph* est inséré dans une *Stage* qui, a priori, est toujours redimensionnable. Le mode Preview est donc particulièrement intéressant pour tester le comportement d'une fenêtre redimensionnable (ce qui n'est pas le cas de la fenêtre "Choix d'une couleur").

3.2. Fichier fxml généré

Et voici un exemple de fichier fxml. On y retrouve la structure du *Scene graph* et la description de tous les composants.

```
<BorderPane maxHeight="-Infinity" maxWidth="-Infinity" minHeight="-Infinity" minWidth="-Infinity"
  prefHeight="300.0" prefWidth="350.0" xmlns="http://javafx.com/javafx/21"
  xmlns:fx="http://javafx.com/fxml/1">
  <top>
    <Label text="Chosissez une couleur :" BorderPane.alignment="CENTER_LEFT">
      <font>
        <Font size="14.0" />
      </font>
    </Label>
  </top>
  <center>
    <Rectangle arcHeight="5.0" arcWidth="5.0" fill="BLUE" height="200.0" stroke="BLACK"
  strokeType="INSIDE" width="220.0" BorderPane.alignment="CENTER" />
  </center>
  <bottom>
    <HBox alignment="CENTER_RIGHT" fillHeight="false" prefHeight="22.0" prefWidth="308.0"
  spacing="20.0" BorderPane.alignment="CENTER_RIGHT">
      <children>
        <Button alignment="CENTER" mnemonicParsing="false" prefHeight="25.0"
  prefWidth="80.0" text="OK" textAlignment="CENTER" />
        <Button alignment="CENTER" maxHeight="-Infinity" maxWidth="-Infinity" minHeight="-Infinity"
  minWidth="-Infinity" mnemonicParsing="false" prefHeight="25.0" prefWidth="80.0"
  text="Annuler" textAlignment="CENTER" />
      </children>
      <padding>
        <Insets bottom="10.0" left="10.0" right="10.0" top="10.0" />
      </padding>
    </HBox>
  </bottom>
  <left>
    <VBox prefHeight="210.0" prefWidth="82.0" spacing="30.0" BorderPane.alignment="CENTER">
      <children>
        <RadioButton mnemonicParsing="false" selected="true" text="Bleu">
          <toggleGroup>
            <ToggleGroup fx:id="groupe" />
          </toggleGroup>
        </RadioButton>
        <RadioButton mnemonicParsing="false" text="Vert" toggleGroup="$groupe" />
        <RadioButton mnemonicParsing="false" text="Jaune" toggleGroup="$groupe" />
        <RadioButton mnemonicParsing="false" text="Orange" toggleGroup="$groupe" />
      </children>
      <padding>
        <Insets bottom="10.0" left="10.0" right="10.0" top="20.0" />
      </padding>
    </VBox>
  </left>
  <padding>
    <Insets bottom="10.0" left="10.0" right="10.0" top="10.0" />
  </padding>
</BorderPane>
```

À FAIRE

Fermez Scene Builder puis ouvrez le fichier fxml avec un éditeur de texte. Dans ce fichier, repérez la balise **<bottom>**, puis la description du "Button" **OK**. Modifiez la valeur de sa propriété **prefWidth** (doublez cette taille par exemple) puis sauvegardez.

Ouvrez à nouveau ce fichier sous Scene Builder.

4. Fenêtre redimensionnable

Dans une fenêtre redimensionnable, une solution simple pour gérer le redimensionnement/repositionnement des composants est d'utiliser une racine de type *AnchorPane*. Il suffit de fixer les distances de chaque composant par rapport à un ou plusieurs bords de la scène. Scene Builder propose un moyen intuitif de fixer ces distances dans le volet Layout :

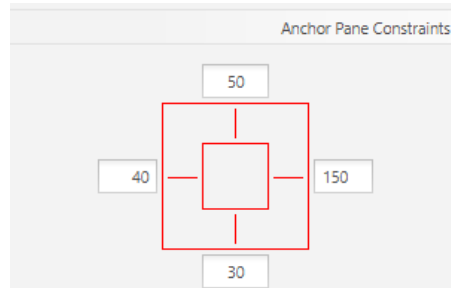
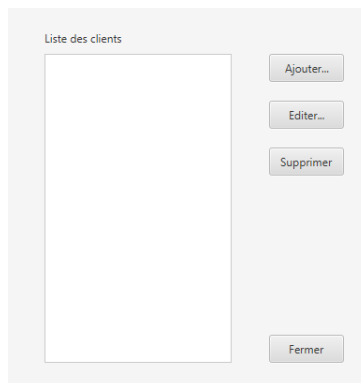


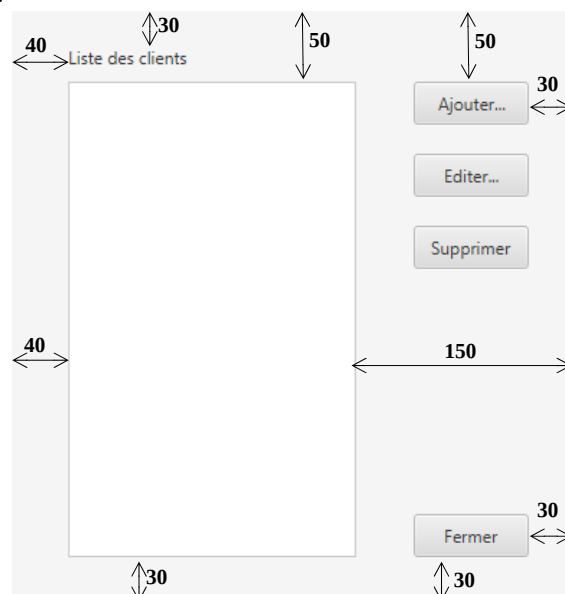
figure 6 : fixer les distances d'un composant dans un AnchorPane

À FAIRE

En choisissant une racine de type *AnchorPane*, réalisez cette maquette en respectant le layout fourni page suivante :



Layout des composants :

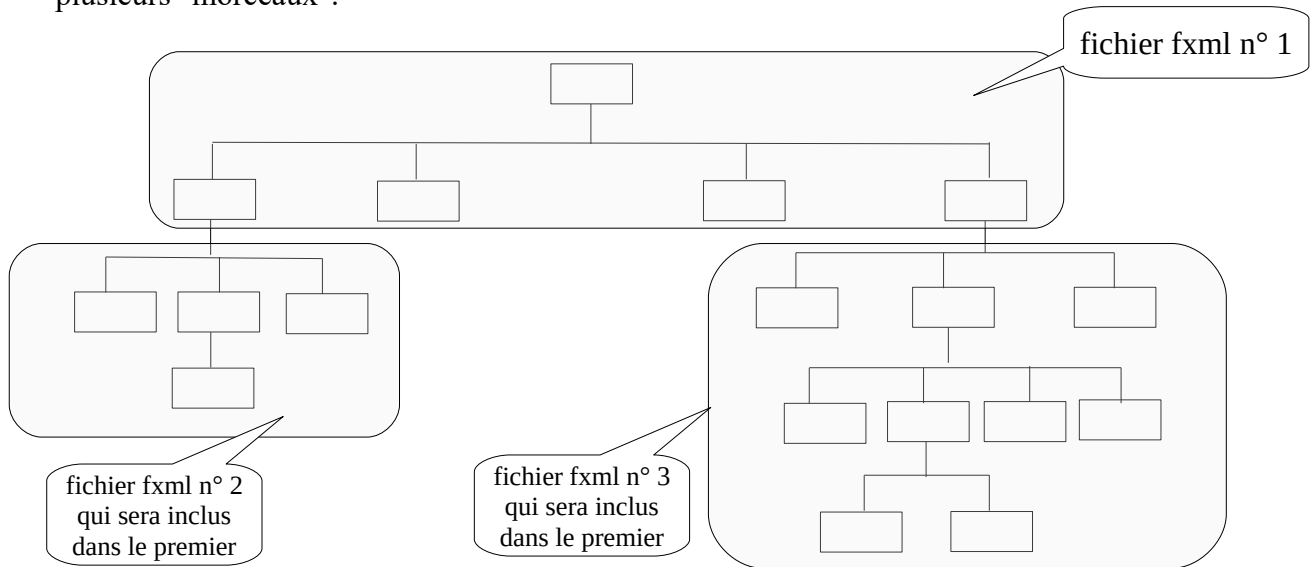


- les boutons ont une taille de 90 x 30
- les trois boutons du haut sont espacés de 20

figure 7 : layout des composants

5. Construction d'une IHM en plusieurs parties

Quand l'IHM à construire est complexe, il est possible de la décomposer en plusieurs parties et de créer un fichier fxml pour chaque partie. Cela revient à construire un Scene graph en plusieurs "morceaux".



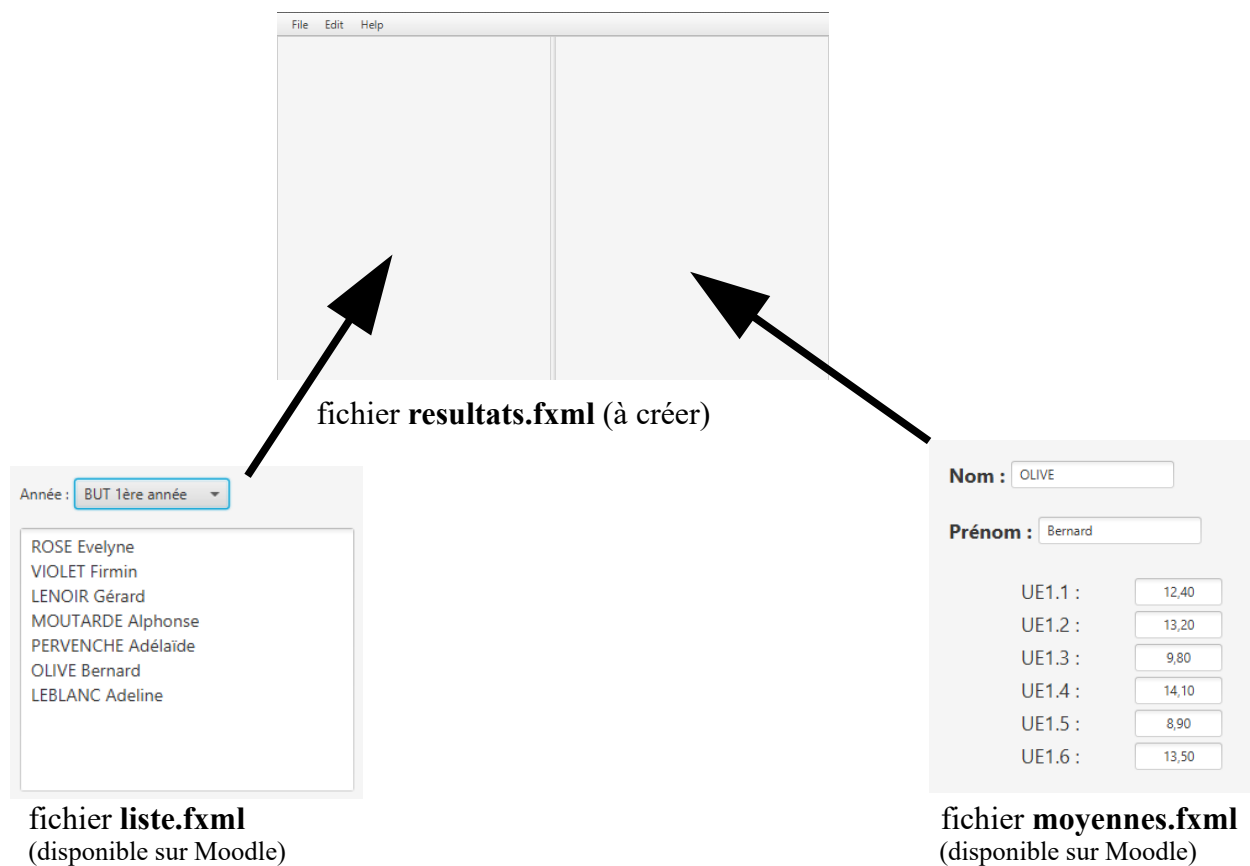
C'est ce que nous allons voir avec cette interface :

L'interface graphique, intitulée "Résultats", est divisée en deux parties principales par une SplitPane. La partie gauche contient une liste de noms : ROSE Evelyne, VIOLET Firmin, LENOIR Gérard, MOUTARDE Alphonse, PERVENCHE Adélaïde, OLIVE Bernard, et LEBLANC Adeline. La partie droite contient des champs de saisie pour "Nom" (OLIVE) et "Prénom" (Bernard), ainsi qu'une liste de six entrées "UE1.1" à "UE1.6" avec des valeurs numériques (12,40, 13,20, 9,80, 14,10, 8,90, 13,50).

Cette IHM se compose d'un panneau principal avec une barre de menus et de deux sous-panneaux présents dans un *SplitPane*².

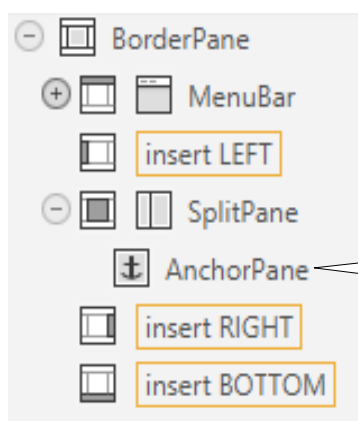
² Un *SplitPane* permet de séparer un panneau en plusieurs zones que l'utilisateur peut redimensionner.

Structure de cette IHM :



À FAIRE

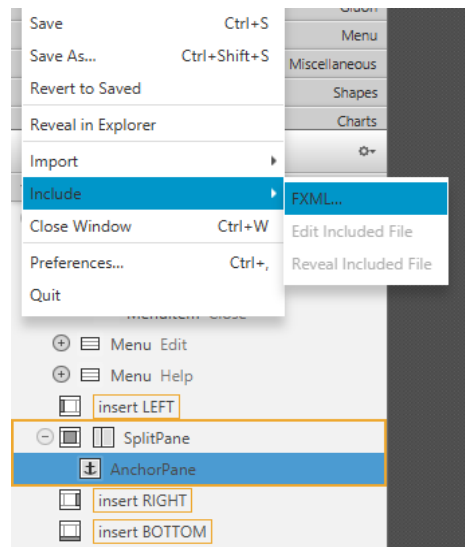
Dans un fichier Résultats.fxml, créez l'IHM principale dont le Scene graph est :



Note : pour pouvoir y inclure d'autres fichiers, le **SPLITpane** doit posséder au moins un "enfant"

Fixez la taille préférée de la racine à 600 x 400.

Une fois l'IHM principale créée, il reste à inclure dans son *Scene graph* les deux autres fichiers fxml : menu **File** → **Include** → **FXML...**



À FAIRE

Dans le *Scene graph* de l'IHM principale, ajoutez au niveau du *SplitPane* les fichiers **liste.fxml** (à gauche) et **moyennes.fxml** (à droite).

Enfin, fixez la largeur minimale de la zone de gauche à 200 et celle de la zone de droite à 300.

Vous pourrez constater la conséquence sur le fichier fxml :

```
<center>
  <SplitPane dividerPositions="0.5" BorderPane.alignment="CENTER">
    <items>
      <fx:include minWidth="200.0" source="liste.fxml" />
      <fx:include minWidth="300.0" source="moyennes.fxml" />
    </items>
  </SplitPane>
</center>
```

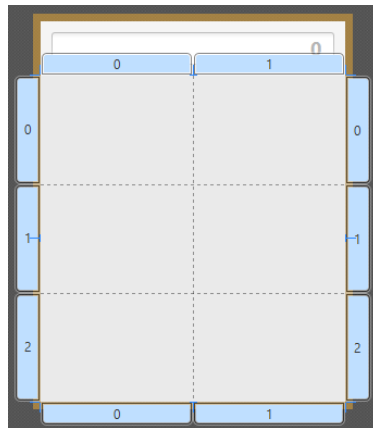
6. Utiliser un GridPane

Le *GridPane* est un peu particulier. Réaliser l'IHM de la calculette va permettre de se familiariser avec l'utilisation de ce Layout Pane.

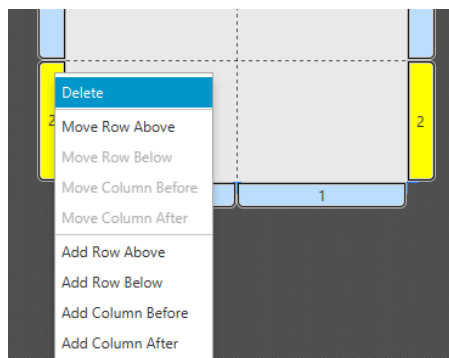
À FAIRE

Créez une nouvelle IHM et ajoutez-lui une racine de type `BorderPane` et, dans sa zone `Top`, un `TextField` avec un `Margin` de 10 tout autour.

Faites glisser ensuite un GridPane dans sa zone Center. Vous devez obtenir ceci :



Par défaut, le GridPane possède 3 lignes et 2 colonnes, mais un clic droit sur la ligne ou la colonne sélectionnée permet d'en supprimer ou d'en ajouter d'autres.



Créez les lignes et colonnes nécessaires pour que le clavier de la calculatrice soit une grille de 4x4. Définissez un padding de 10. Enfin, fixer un HGap et un VGap de 10.

Les cellules du GridPane contiennent le même type de composant, le clavier de la calculatrice est en effet composé de 16 boutons identiques. Il faut alors créer un Button et le définir complètement avant de le dupliquer dans les autres cellules.

Note : il est préférable que la définition du bouton soit complète avant sa duplication.

À FAIRE

Faites glisser un Button dans la première cellule et définissez les propriétés :

- texte du bouton : centré*
- texte du bouton : police "System" et de taille 18*

puis définissez son Layout :

- HGrow et VGrow : ALWAYS*
- dimension minimale : 20 x 20*
- dimension maximale : MAX_VALUE*

Dupliquez ensuite ce bouton dans les autres cellules (clic droit sur le bouton, puis Duplicate, puis faire glisser la copie dans une autre cellule).

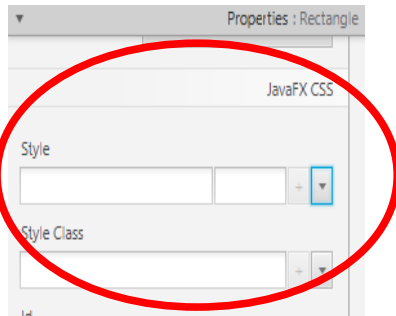
Testez le comportement de la fenêtre en mode Preview.

7. Un peu de CSS

7.1. Appliquer un style

Si l'apparence par défaut d'un composant n'est pas satisfaisante, on peut modifier son aspect en lui appliquant des styles, c'est-à-dire en modifiant la valeur de ses attributs graphiques.

C'est dans la section *JavaFX CSS* du volet *Properties* du composant que son style peut être modifié :



À FAIRE

Reprenez l'IHM "**Couleur**" et modifiez l'apparence du rectangle en lui appliquant les styles suivants :



Note : cliquer dans la zone *Style* fait apparaître la liste des attributs graphiques du composant ; cliquer sur + permet d'ajouter un nouveau style.

Note : la documentation concernant le CSS avec JavaFX se trouve sur :

<https://docs.oracle.com/javase/8/javafx/api/javafx/scene/doc-files/cssref.html>

7.2. Les feuilles de style CSS

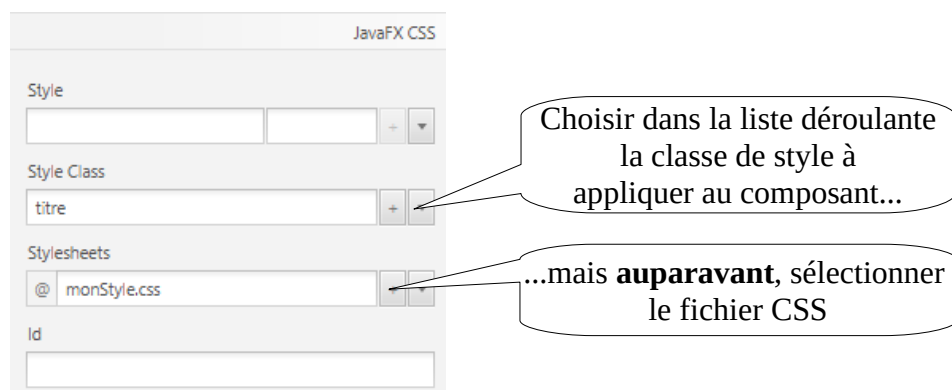
Appliquer un style comme précédemment devient vite assez fastidieux. Une meilleure solution consiste à définir, dans un fichier CSS, des classes de style qu'on associera aux composants. Le fichier *monStyle.css*, disponible sur Moodle, contient trois classes de style :

```
.bouton{
  -fx-background-color: brown;
  -fx-font-family: "Arial";
}

.titre{
  -fx-font-family: "Arial";
  -fx-font-size: 16;
  -fx-font-weight: bold;
}

.zoneBoutons{
  -fx-background-color: grey;
}
```

On peut alors appliquer une classe de style à un composant :



À FAIRE

Toujours sur l'IHM "Couleur", appliquez :

- la classe de style **.bouton** aux boutons OK et Annuler,
- la classe de style **.titre** au label de l'entête,
- et la classe de style **.zoneBoutons** à la HBox.